

**UNIVERSIDAD POLITÉCNICA DE
SAN LUIS POTOSÍ**

CARRERA: Ingeniería en Tecnologías de la Información

MATERIA: CNO IV Seguridad Informática

TRABAJO: FILE PATH TRAVERSAL, VALIDATION OF FILE EXTENSION
WITH NULL BYTE BYPASS

Ojeda Camacho José Carlos / 181629

PROFESOR: Servando Contreras

Contenido

Información del laboratorio	2
Objetivo del laboratorio.....	2
Explicación técnica de la vulnerabilidad	2
Procedimiento paso a paso	3
Explicación del payload	5
Mitigación recomendada.....	5

Información del laboratorio

- **Nombre del laboratorio:** FILE PATH TRAVERSAL, VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS
- **Nivel (Apprentice / Practitioner):** Practitioner
- **Tipo de explotación:** PATH TRAVERSAL / BYPASS DE VALIDACIÓN DE EXTENSIÓN

Objetivo del laboratorio

El objetivo de este laboratorio es explotar una vulnerabilidad de path traversal en la funcionalidad de visualización de imágenes de productos de una aplicación web, la cual exige de manera estricta que los archivos solicitados terminen con una extensión específica (como .png). Para resolverlo, se debe evadir esta validación lógica utilizando un byte nulo (%00) que permita burlar el filtro del backend y lograr extraer con éxito el contenido del archivo crítico del sistema operativo, /etc/passwd.

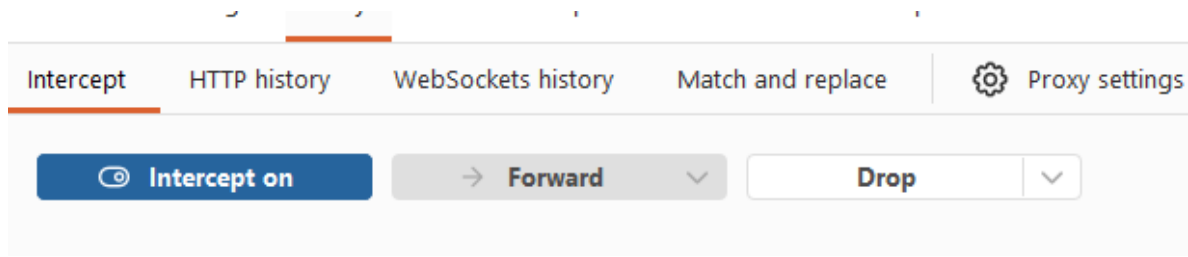
Explicación técnica de la vulnerabilidad

La vulnerabilidad se origina en una deficiencia en la validación de las entradas y en la disparidad en cómo se procesan las cadenas de texto entre la lógica de la aplicación web y el sistema operativo subyacente. La aplicación implementa una validación basada en la extensión del archivo, bloqueando cualquier solicitud que no finalice con un tipo de archivo permitido. Sin embargo, si el backend está desarrollado en lenguajes o entornos donde las funciones de manejo de archivos de bajo nivel (escritas en C/C++) interpretan un byte nulo (\0 o %00 en formato URL) como el carácter de fin de cadena (null terminator), es posible truncar la validación de la extensión. De este modo, la aplicación valida la presencia de la extensión requerida al final de la cadena, pero el sistema operativo lee únicamente los caracteres previos al byte nulo, permitiendo la lectura de archivos arbitrarios del sistema mediante técnicas de salto de directorio (*path traversal*).

Procedimiento paso a paso

El proceso inicia con la apertura de la aplicación Burp Suite y el uso de su navegador integrado, el cual evita conflictos de configuración con certificados de seguridad.

Posteriormente, en la sección **Proxy > Intercept**, se verifica que la función de intercepción se encuentre activada (Intercept is on). Esto permite pausar y capturar de manera temporal las peticiones HTTP generadas por el navegador antes de que alcancen el servidor de la aplicación, facilitando así su análisis y posterior modificación.



Una vez que la intercepción está activa y la página carga los elementos de la tienda, se interactúa con cualquiera de los productos para visualizar las imágenes. Las peticiones de carga de imágenes se reflejan en el historial de tráfico HTTP, identificando parámetros del tipo /image?filename=1.jpg.

Haciendo clic derecho sobre la petición capturada en el historial del proxy o en la sección de intercepción, se despliega el menú contextual y se selecciona la opción **Send to Repeater**. Esta acción traslada la petición a la herramienta Repeater de Burp Suite, la cual permite reenviar de forma manual y controlada tantas solicitudes como sea necesario, facilitando la prueba de iteraciones y modificaciones en los parámetros sin requerir una nueva interacción directa desde el navegador web.

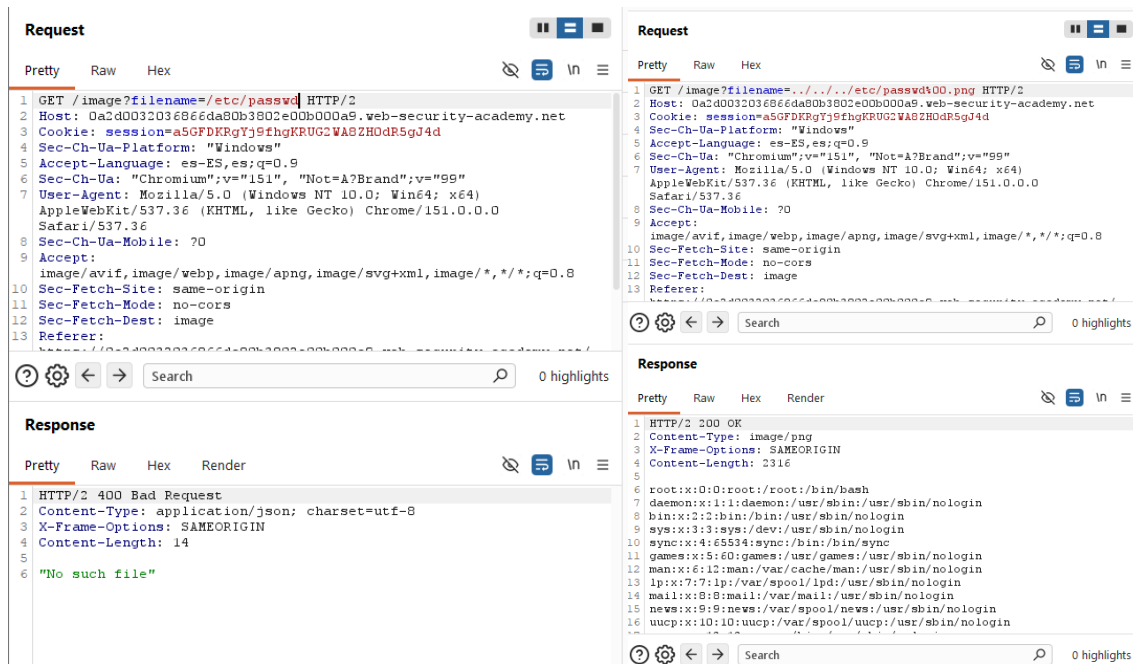
Time	Type	Direction	Method	URL	Stat
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=37.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=31.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=51.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=23.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=43.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=R.inn	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	
21:01:4...	HT...	→	Request	GET https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/image?filename=8.jpg	

Request
Pretty Raw Hex
1 GET /image?filename=37.jpg HTTP/2
2 Host: 0a2d0032036866da80b3802e00b000a9.web-security-academy.net
3 Cookie: session=a50fD8KpY5f5hgKRUG:WAS2H0dR5gJ4d
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif, image/webp, image/apng, image/svg+xml, image/*, */*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a2d0032036866da80b3802e00b000a9.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 ...

Request cookies 1
Request headers 17

Dentro de la herramienta Repeater, se procede a modificar los parámetros de la petición HTTP capturada. Específicamente, en la solicitud del parámetro filename, se introduce una secuencia de path traversal combinada con el archivo objetivo (`../../etc/passwd`). No obstante, al añadir la secuencia estándar, la aplicación devuelve un error debido a que el archivo no finaliza con la extensión permitida. Para solucionar esta restricción, se aplica la técnica de **bypass por byte nulo**, modificando el valor del parámetro de la siguiente manera:

filename=../../etc/passwd%00.pngc



Al enviar la solicitud modificada desde el Repeater, se observa que el servidor la procesa correctamente y devuelve una respuesta con el código de estado HTTP/2 200 OK. En el cuerpo de la respuesta HTTP, el servidor expone el contenido íntegro del archivo `/etc/passwd` solicitado, mostrando claramente los registros de usuarios del sistema (tales como `root`, `daemon`, `bin`, entre otros). Esto confirma la explotación exitosa de la vulnerabilidad y resuelve de manera automática el laboratorio en la plataforma de PortSwigger.

Explicación del payload

El payload utilizado en este laboratorio consiste en una cadena compuesta por tres elementos clave:

1. **Secuencias de salto de directorio (../..../):** Permiten navegar hacia atrás en la jerarquía de directorios del servidor para salir de la carpeta raíz de imágenes y posicionarse en la raíz del sistema de archivos.
2. **El archivo objetivo (etc/passwd):** La ruta absoluta hacia el archivo crítico del sistema Unix/Linux que almacena la información de las cuentas de usuario.
3. **El byte nulo y la extensión requerida (%00.png):** El byte nulo (%00) actúa como un carácter de fin de cadena a nivel del sistema operativo backend, lo que provoca que el sistema ignore todo lo que se encuentre después de este (en este caso, .png). Sin embargo, el filtro de validación de la aplicación web procesa la cadena completa y detecta la presencia del .png exigido, permitiendo que la solicitud pase el control de seguridad de manera fraudulenta.

Mitigación recomendada

Para prevenir vulnerabilidades de Path Traversal y eludir controles de extensión mediante bytes nulos, es necesario implementar un control riguroso sobre la validación y el procesamiento de las entradas del usuario. Las principales medidas de mitigación incluyen:

- **Actualización del entorno de desarrollo:** Utilizar lenguajes de programación modernos (como Java, C#, o versiones recientes de PHP) y funciones nativas que no sufran de vulnerabilidades de truncamiento por bytes nulos al interactuar con el sistema de archivos.
- **Uso de funciones de mapeo seguro (Basename):** Utilizar funciones integradas que extraigan únicamente el nombre base del archivo solicitado, descartando por completo cualquier ruta relativa, absoluta o caracteres especiales contenidos en la entrada.
- **Validación mediante listas blancas (Whitelisting):** Restringir los valores permitidos en los parámetros de entrada a un conjunto estricto de nombres de archivos válidos o identificadores predefinidos, rechazando de manera absoluta cualquier cadena que contenga caracteres nulos (%00), secuencias de escape (../) o barras.
- **Directorio raíz confinado (Chroot o aislamiento):** Asegurar que la aplicación web se ejecute con privilegios mínimos y que el acceso al sistema de archivos esté estrictamente confinado a un directorio específico, impidiendo que el proceso lea fuera de esa estructura autorizada.